

CS536 Science of Programming
Fall 2025
Assignment 3 Sample Solution Sketches

1. (a) True. If $\sigma(p) = \perp$, then $\sigma \not\models p$ and p is not valid.
 (b) False. It is possible that $\tau(p) = \perp$.
 (c) True. Since $\Sigma_0 \models p$, then $\forall \sigma \in \Sigma_0. \sigma \models p$. We also have $\tau \models p$, so $\forall \sigma \in \Sigma_0 \cup \{\tau\}. \sigma \models p$; thus, $\Sigma_0 \cup \{\tau\} \models p$.
 (d) True. No matter what p is, we always have that each state in $\emptyset$ satisfies p .
 (e) False. It is possible that some states in Σ_0 satisfy $x > 1$ and others satisfy $x \leq 1$; thus, we cannot say that the entire collection Σ_0 satisfies $x > 1$ or $x \leq 1$.
2. (a) All four guards are satisfied, so IF can choose each arm non-deterministically to execute. Recall that the result of division is rounded towards 0.

$$M(IF, \sigma_1) = \{\{x = 2, y = 1\}, \{x = 3, y = 2\}, \{x = 0, y = 1\}, \{x = 3, y = 1\}\}$$

 (b) With $\sigma_2(x) = \sigma_2(y) = 1$, no guard is satisfied. Thus, $M(IF, \sigma_2) = \{\perp_e\}$.
 (c) Here, I only show the collection of states after each iteration.

$$\begin{aligned}
 & M(DO, \sigma_1) \\
 &= M(DO, \{\{x = 2, y = 1\}, \{x = 3, y = 2\}, \{x = 0, y = 1\}, \{x = 3, y = 1\}\}) \\
 &= M(DO, \{\{x = 1, y = 1\}, \{x = 2, y = 2\}, \{x = 3, y = 3\}, \{x = 0, y = 1\}, \perp_d\}) \\
 &= M(DO, \{\{x = 1, y = 1\}, \{x = 1, y = 2\}, \{x = 3, y = 3\}, \{x = 0, y = 1\}, \perp_d\}) \\
 &= \{\{x = 1, y = 1\}, \{x = 1, y = 2\}, \{x = 3, y = 3\}, \{x = 0, y = 1\}, \perp_d\}
 \end{aligned}$$

After the first iteration, states $\{x = 2, y = 1\}$, $\{x = 3, y = 2\}$ and $\{x = 3, y = 1\}$ satisfies some guards of the loop. We also noticed that the state $\{x = 3, y = 1\}$ appears again, so we can say that it will be a divergence.

After the second iteration, only the state $\{x = 2, y = 2\}$ can pass some guards for the next iteration. Running time loop with $\{x = 2, y = 2\}$ for another iteration, then we get states $\{x = 2, y = 1\}$ and $\{x = 1, y = 2\}$. The state $\{x = 1, y = 2\}$ is already one of the possible states on which DO could terminate, and the state $\{x = 2, y = 1\}$ appears in previous iterations, so it will be a divergence.

- (d) With $\sigma_2(x) = \sigma_2(y) = 1$, no guard is satisfied. Thus, $M(DO, \sigma_2) = \{\sigma_2\}$.

3.

$MAJORITY \equiv k_0 = 0; k_1 = 0;$
 $\quad \mathbf{while} \ k_0 < n \wedge k_1 < n \ \mathbf{do} \ J; k_0 := k_0 + 1; k_1 := k_1 + 1 \ \mathbf{od};$
 $\quad \mathbf{if} \ k_1 = n \rightarrow major := 0 \ \square \ k_0 = n \rightarrow major := 1 \ \mathbf{fi}$

Here, $J \equiv \mathbf{do} \ b[k_0] = 1 \rightarrow k_0 := k_0 + 1 \ \square \ b[k_1] = 0 \rightarrow k_1 := k_1 + 1 \ \mathbf{od}$.

After the deterministic while loop, we must have $k_0 = n$ or $k_1 = n$, so there will be no runtime error in the execution of the non-deterministic conditional statement. Actually, I think k_0 and k_1 cannot equal n at the same time, so a deterministic conditional statement can be used here as well.

There are (should be?) other ways to implement this program. The key is to pair one 0 and one 1 in each iteration without missing any possible pairs in one (or several) scans.

4. (a) False. A nondeterministic program could terminate in one state as well.
 (b) True. If $\sigma \not\models \{p\}S\{q\}$, then $\sigma \models p$ and $M(S, \sigma) - \perp \not\models q$.
 (c) False. If $\sigma \models_{tot} \{p\}S\{q\}$, then $\sigma \not\models p$ or $M(S, \sigma) \models q$; it is not necessarily true that $\sigma \not\models p$.
 (d) False. If $\sigma \not\models \{p\}S\{q\}$, then $\sigma \models p$ and $M(S, \sigma) - \perp \not\models q$. We cannot get $M(S, \sigma) \models q$.
 (e) True. If $\sigma \not\models \{p\}S\{q\}$, then $\sigma \models p$ and $M(S, \sigma) - \perp \not\models q$, which implies that $M(S, \sigma) \not\models q$ and thus $\sigma \not\models_{tot} \{p\}S\{q\}$.
5. (a) To have $\perp_e \notin M(S, \sigma)$, we need state σ with:
 - $\sigma(x) \geq 0$ to avoid “square root of negative number” error;
 - $0 \leq \sigma(x) < 4$ to avoid “array index out of bound” error;
 - and $\sigma(x) \neq 1$ to avoid “division by 0” error.
 Thus, the possible values of $\sigma(x)$ are 0, 2, 3.
 (b) So that $\sigma \models \{sqrt(x) \neq 1\} x := 1/x \{F\}$, we can have:
 - $\sigma \not\models sqrt(x) \neq 1$, thus $\sigma(sqrt(x)) = 1$ and $\sigma(x) = 1$.
 - $\sigma \models sqrt(x) \neq 1$ and $M(x := 1/x, \sigma) - \perp \models F$, thus $M(x := 1/x, \sigma) - \perp = \emptyset$ and $\sigma(x) = 0$.
 Thus, the possible values of $\sigma(x)$ are 0 and 1.
 (c)
 - When $\sigma(x) = 0$, the precondition is not satisfied, so the triple is satisfied.
 - When $\sigma(x) < 0$ or when $\sigma(x)$ is odd, the program will diverge; which is acceptable for partial correctness.
 - When $\sigma(x) > 0$ and $\sigma(x)$ is even, the program will terminate in some state τ with $\tau(x) = 0$, which does not satisfy the post-condition.
 Thus, for satisfaction under partial correctness, we need $\sigma(x) \leq 0$ or $\sigma(x) \% 2 = 1$.
 (d) For total correctness, the program has to terminate in σ ; so the only possible value for $\sigma(x)$ is 0.
6. (a) True. A deterministic statement cannot terminate in more than one states.
 (b) False. We know that $\sigma \models T$ and $M(S, \sigma) \not\models q$, thus $\sigma \not\models_{tot} \{T\}S\{q\}$ and the triple is not valid under total correctness.
 (c) True. Since $\sigma \models p$, thus $\sigma \not\models \neg p$. Thus, σ satisfies the triple trivially.
 (d) False. We know that $\sigma \models p$ and $M(S, \sigma) \not\models F$, thus $\sigma \not\models_{tot} \{p\}S\{F\}$ and the triple is not valid under total correctness.
 (e) False. We know that $\sigma \models p$ and $M(S, \sigma) - \perp \not\models F$, thus $\sigma \not\models \{p\}S\{F\}$ and the triple is not valid under partial correctness.

7. (a) True, it follows immediately from the backward assignment rule.
 We can also argue its validity without using the backward assignment rule: the pre-condition is logically equivalent to $P(k, s+1) \wedge P(k, x+1)$, where variable x is a copy of s . The program increases s by 1, and does not update x , so we know that $P(k, x+1)$ should be true in the post-condition. Since $s = x+1$ after s being increased, so $P(k, s)$ is true in the post-condition.
- (b) False. A witness can be $\sigma = \{s = 0, k = 0\}$. We have $\sigma \models P(0, 0)$, but $M(s := s+1, \sigma) = \{s = 1, k = 0\} \not\models P(0, 1)$.
- (c) True. The precondition cannot be satisfied by any state.
- (d) True. The precondition logically implies $P(k, s) \wedge P(k, x)$. The program does not update x , so $P(k, x)$ in the post-condition is still true.
- (e) True, using backward assignment we can get (both partial and totally) valid triples: $\{P(k+1, s)\}k := k+1\{P(k, s)\}$ and $\{P(k+1, s+1)\}s := s+1\{P(k+1, s)\}$. Combine them using the sequence rule, and we get: $\{P(k+1, s+1)\}s := s+1; k := k+1\{P(k, s)\}$.
 Similar to (a), we can also argue its validity without using any rules here.
8. (a) Yes. If $\sigma \models p_1 \wedge p_2$, then $\sigma \models p_1$ and $\sigma \models p_2$. Since $\sigma \models_{tot} \{p_1\}S\{q_1\}$ and $\sigma \models_{tot} \{p_2\}S\{q_2\}$, thus $M(S, \sigma) \models q_1$ and $M(S, \sigma) \models q_2$; which logically implies that $M(S, \sigma) \models q_1 \wedge q_2$. Thus, $\sigma \models_{tot} \{p_1 \wedge p_2\}S\{q_1 \wedge q_2\}$.
- (b) Yes. If $\sigma \models p_1 \vee p_2$, then $\sigma \models p_1$ or $\sigma \models p_2$. Since $\sigma \models_{tot} \{p_1\}S\{q_1\}$ and $\sigma \models_{tot} \{p_2\}S\{q_2\}$, thus $M(S, \sigma) \models q_1$ or $M(S, \sigma) \models q_2$; which logically implies that $M(S, \sigma) \models q_1 \vee q_2$. Thus, $\sigma \models_{tot} \{p_1 \vee p_2\}S\{q_1 \vee q_2\}$.
- (c) Yes. The post-condition of the triple semantically equals to $\neg q_1 \vee q_2$, and $q_2 \Rightarrow \neg q_1 \vee q_2$. Since $\sigma \models_{tot} \{p_2\}S\{q_2\}$, thus $\sigma \models_{tot} \{p_2\}S\{q_1 \rightarrow q_2\}$ (weaken the post-condition).